Apparel Distribution ERP

System Design and Technical Documentation

Multi-brand men and women clothing distribution

Stack: MongoDB - Express - React - Node.js (MERN)

Document details

Version: 1.0 (design draft)

Date: October 2026

Status: for review before development starts

Prepared with: Claude (AI-assisted draft, review with your team)

Contents

Contents 1

1. Project overview 2

2. User roles 3

3. Access control matrix 4

4. System architecture 5

5. Functional modules 6

6. Database design 8

7. Security design 10

8. API design and project structure 11

9. Key business rules 12

10. Build roadmap, testing and deployment 12

11. Risks and glossary 13

1. Project overview

This document describes a web-based ERP (Enterprise Resource Planning) system for a company that distributes clothing from multiple brands, for men and women, through a network of parties (dealers and retailers). One connected system replaces separate Excel sheets, notebooks and WhatsApp messages, so that employee data, brands, targets, bookings, invoices and payments all share the same information.

1.1 Goals

- Give management one place to run employees, salaries, brands, sub-brands, parties, bookings and targets.
- Show brand-wise, party-wise and employee-wise sales against targets in real time.
- Control who can see and change what, with three Super Admins and any number of Admins.
- Give every salesperson a simple view of their own parties, bookings, targets and salary slips.
- Keep a tamper-resistant history of every important change (audit log).

1.2 Key requirements (from the project owner)

Requirement	How the system covers it
Employee management: profile, salary, managed brands, sales, sub-brands, target achieved	Employee module (section 5.1), brand assignment mapping, salary module, target and achievement engine
Brand-wise sales, target, achieved	Brand and Target modules, Reports (section 5.9)
Party details; which party sells which brand	Party module with Party-Brand mapping (section 5.4)
Booking details	Booking module with size-color line items and status workflow (section 5.5)
Multiple admins and three Super Admins	Role model and permission matrix (sections 3 and 4)
Everything managed by admin	Admin and Super Admin consoles with scoped permissions

1.3 Out of scope for version 1

- Full GST filing (the system prepares GST-ready invoices and a Tally export, filing stays with the accountant).
- Manufacturing or production planning.
- A native mobile app (version 1 uses a responsive web app / PWA that works on phones).

2. User roles

The system has three role levels. Admin permissions are configurable, so one Admin can run HR while another runs sales for specific brands.

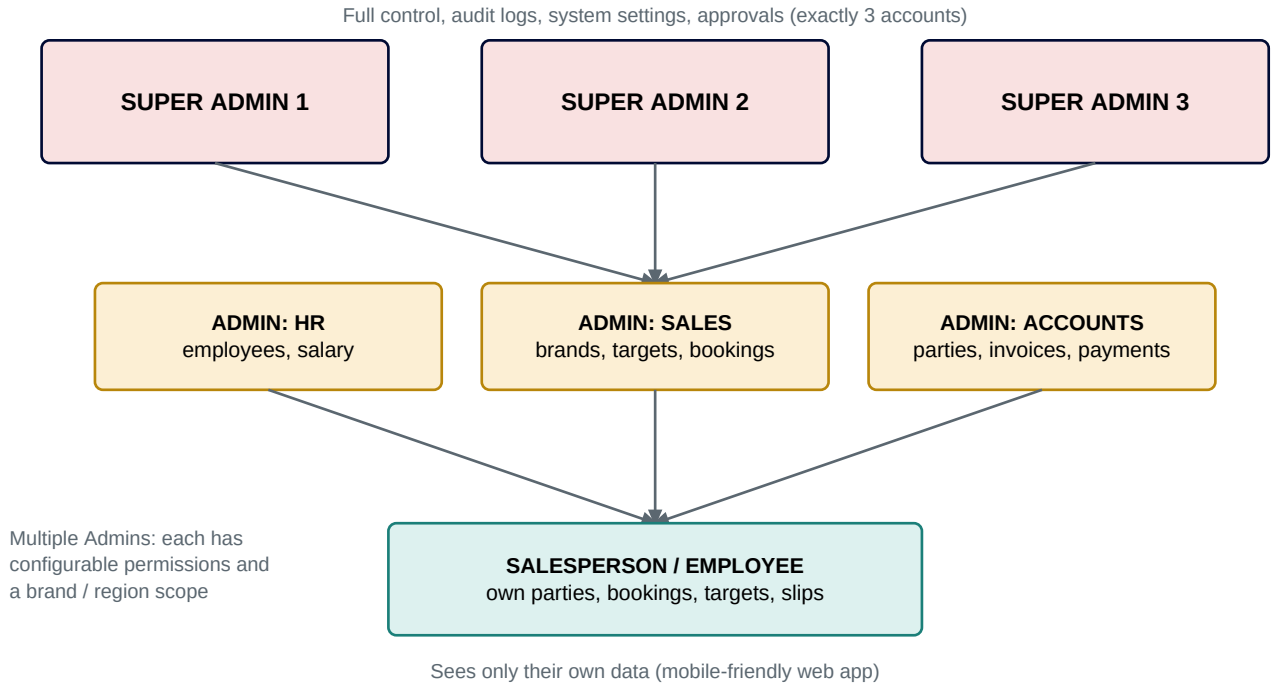

Figure 1 - Role hierarchy

Role	Count	Purpose	Created by
Super Admin	Exactly 3	Owns the system: creates Admins, sees all data, changes settings, approves sensitive actions, reads audit logs.	Seeded at install; only another Super Admin can change this list
Admin	Many	Runs daily work inside the modules and the brand/region scope granted to them.	Super Admin
Salesperson / Employee	Many	Takes bookings, tracks own targets, views own parties and salary slips.	Admin (HR) or Super Admin

Recommended safeguard for three Super Admins: require 2 of 3 Super Admins to approve the highest-risk actions (creating or deleting a Super Admin, changing the audit settings, bulk deleting data). This is a design suggestion; you can relax it to one approval if it feels too strict for your company.

3. Access control matrix

Legend: **C** create, **R** read, **U** update, **D** delete (soft delete), **A** approve. "Scoped" means limited to the brands, regions or parties assigned to that Admin. "Own" means only the user's own records. Admin cells show the *maximum*; a Super Admin can grant less per Admin.

Module / feature	Super Admin	Admin (scoped)	Salesperson
Admin accounts and permissions	C R U D	-	-
Super Admin list and system settings	R U (2-of-3 approval)	-	-
Audit logs	R (read-only)	-	-
Employee profiles and documents	C R U D	C R U (D if granted)	R own, U contact details
Salary structure, advances, slips	C R U D A	C R U if HR permission	R own slips
Attendance and leave	C R U D A	C R U D A	C check-in, C leave request, R own
Brands and sub-brands	C R U D	C R U (scoped)	R assigned only
Products and stock (size / color)	C R U D	C R U (scoped)	R assigned brands
Parties (dealers / retailers)	C R U D	C R U (scoped)	R assigned; C request (needs approval)
Party-Brand mapping	C R U D	C R U (scoped)	R own parties
Bookings / orders	C R U D A	C R U D A (scoped)	C, R own, U before confirmation
Targets	C R U D A	C R U (edits need approval)	R own
Invoices and credit notes	C R U D	C R U (scoped)	R own parties
Payments and outstanding	C R U D	C R U (accounts permission)	R own parties (balance only)
Discount and credit-limit approvals	A (any value)	A (up to set limit)	Request only
Reports and dashboards	All company data	Scoped data	Own performance
Data export (Excel/PDF), Tally export	Yes	If permission granted	No
Backups and restore	Yes	-	-

3.1 How permissions are stored

Each Admin has a permission document with one entry per module and one flag per action, plus a scope. Roles are never hard-coded in screens; every API call checks the permission document, so access can be changed without new code.

```
admin.permissions = {
  employees: { view: true,  add: true,  edit: true,  delete: false },
  salary:    { view: true,  add: false, edit: false, delete: false },
  bookings:  { view: true,  add: true,  edit: true,  delete: false, approveUpTo: 50000 },
  ...
}
admin.scope = { brandIds: [ ... ], regions: ["Patna", "Gaya"], partyIds: [] }
```

4. System architecture

A standard layered MERN architecture: React clients talk to a Node/Express REST API over HTTPS; the API applies security middleware, runs business rules in a service layer, and stores data in MongoDB. Redis and background workers handle caching, rate limits, PDF generation and notifications.

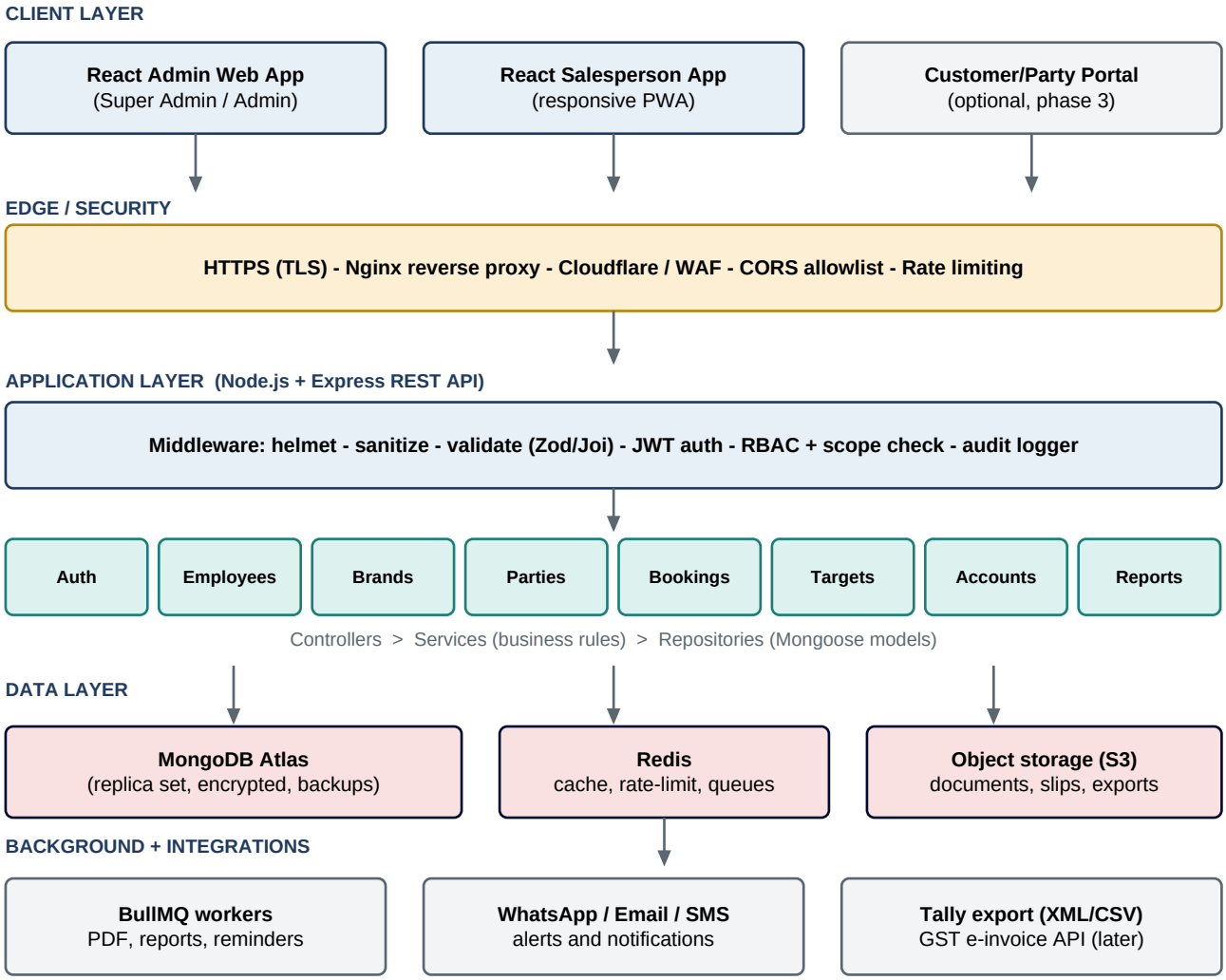

Figure 2 - Layered architecture

4.1 Technology stack

Layer	Choice	Why
Frontend	React 18 + Vite, React Router, TanStack Query, Tailwind CSS, React Hook Form + Zod	Fast builds, clean data fetching/caching, easy forms and validation
UI components	shadcn/ui or MUI; Recharts for charts; TanStack Table for data grids	Admin-style tables and dashboards out of the box
Backend	Node.js (LTS) + Express, TypeScript recommended	Typed code prevents many ERP calculation bugs
Database	MongoDB Atlas (replica set) with Mongoose	Flexible variants (size/color), transactions on replica set
Cache / queue	Redis + BullMQ	Rate limiting, report caching, background jobs
Auth	JWT access token + rotating refresh token, argon2/bcrypt, TOTP 2FA	Stateless API, short-lived access
Files	AWS S3 (or compatible) with signed URLs	Employee documents, slips, exports kept off the DB
PDF / Excel	Puppeteer or pdfmake; ExcelJS	Invoices, salary slips, report exports

Layer	Choice	Why
Notifications	WhatsApp Business API provider, SMTP email	Payment reminders, booking updates
DevOps	Docker, Nginx, GitHub Actions CI/CD, PM2 or container host	Repeatable deployments
Monitoring	Sentry (errors), Winston/Pino logs, uptime checks	Find problems before users report them
Testing	Jest/Vitest, Supertest, Playwright	Unit, API and end-to-end tests

Honest note on MongoDB: ERP data is highly relational (parties, bookings, invoices, payments) and money must be exact. MongoDB works well if you (1) use a replica set so multi-step actions run in transactions, (2) store money as integers in paise, and (3) enforce relations in the service layer. A SQL database such as PostgreSQL is a strong alternative if you ever outgrow this; your choice of MERN is fully workable for this project.

5. Functional modules

5.1 Employee management

- Profile: name, photo, contact, address, joining date, designation, reporting manager, status.
- Documents: ID proof, offer letter, bank details (stored encrypted); store only the last 4 digits of Aadhaar.
- Salary structure: basic, allowances, deductions, PF/ESI, advances, incentives; monthly salary run and PDF salary slips.
- Attendance and leave: check-in/out, leave balance, leave requests with approval.
- Brand and sub-brand assignment: which brands each employee manages or sells.
- Performance view: sales, target, achievement percentage and incentive earned, per month and season.

5.2 Brand and sub-brand management

- Hierarchy: Brand > Sub-brand (for example men's line, women's line, kids) > Products.
- Link employees and parties to brands for scope control and reporting.
- Brand-level settings: GST rate, default discount, season calendar.

5.3 Products and stock

- Item master with style code, size, color, season, MRP, dealer price and barcode/SKU.
- Size-color matrix stock view per warehouse; stock reserved on booking, reduced on invoice.
- Low-stock and dead-stock alerts.

5.4 Party (dealer / retailer) management

- Details: name, city, region, GST number, contact persons, credit limit, payment terms, outstanding balance.
- Party-Brand mapping: which party sells which brands, with brand-wise price list and discount.
- Assigned employee, purchase history and brand-wise sales per party.
- New party requests from salespeople go through admin approval.

5.5 Booking / order management

- Booking entry by party, brand, sub-brand, style, size, color, quantity, rate, discount and delivery date.
- Status flow: Draft > Booked > Confirmed > Dispatched > Delivered (or Cancelled).
- Checks before confirmation: stock available, credit limit not exceeded, discount within limit (else approval request).
- Booking vs actual sales comparison and pending-booking report.

5.6 Targets and achievement

- Targets per employee, per brand/sub-brand, per month, season or year, in value and quantity.
- Achievement is calculated automatically from confirmed bookings and invoices (rule set in section 9).
- Incentive slabs linked to achievement percentage; target edits are approval-gated and logged.

5.7 Invoicing, accounts and outstanding

- GST-ready invoices, challans, returns and credit notes; e-invoice / e-way bill support in a later phase.
- Payments received, party ledger, outstanding with 30/60/90-day ageing, credit-limit control.
- Automatic payment reminders by WhatsApp or email.
- Tally export of sales, purchase and receipts.

5.8 Approvals and notifications

- Configurable approval rules: discount above limit, credit-limit exception, target edit, salary change, new party.
- In-app notifications plus WhatsApp/email for target shortfall, payment due, booking dispatched, approval pending.

5.9 Reports and dashboards

- Super Admin dashboard: company sales, brand-wise sales vs target, top and slow parties, outstanding, employee leaderboard.
- Admin dashboard: same, limited to scope. Salesperson dashboard: own target, bookings, parties, incentive.
- Filters: date, season, brand, sub-brand, region, employee, party. Export to Excel and PDF.

5.10 Extra features suggested for later phases

Feature	Benefit	Phase
Salesperson mobile PWA with offline booking and photo proof of visit	Field staff can work without constant internet	2
WhatsApp invoice and reminder sending	Faster collection, fewer calls	2
Season management (autumn/winter, spring/summer)	Season-aware targets, stock and reports	2
Beat / visit planning and target vs visit tracking	Better field discipline	3
Party portal (dealers view own orders, ledger, catalogue)	Fewer calls to the office	3
Claude-powered bill and document reader (purchase bills to entries)	Removes manual typing for accounts	3
AI summary of daily/weekly sales for owners (sent on WhatsApp)	One-message business overview	3
Multi-warehouse transfers and barcode scanning	Accurate stock	3
Data import tool (Excel to system) for onboarding a company	Faster go-live for each client	1-2

6. Database design

Figure 3 shows the main collections and how they relate. Link lines show cardinality (1:1, 1:N, N:1). The two mapping collections (EmployeeBrand, PartyBrand) implement many-to-many relationships.

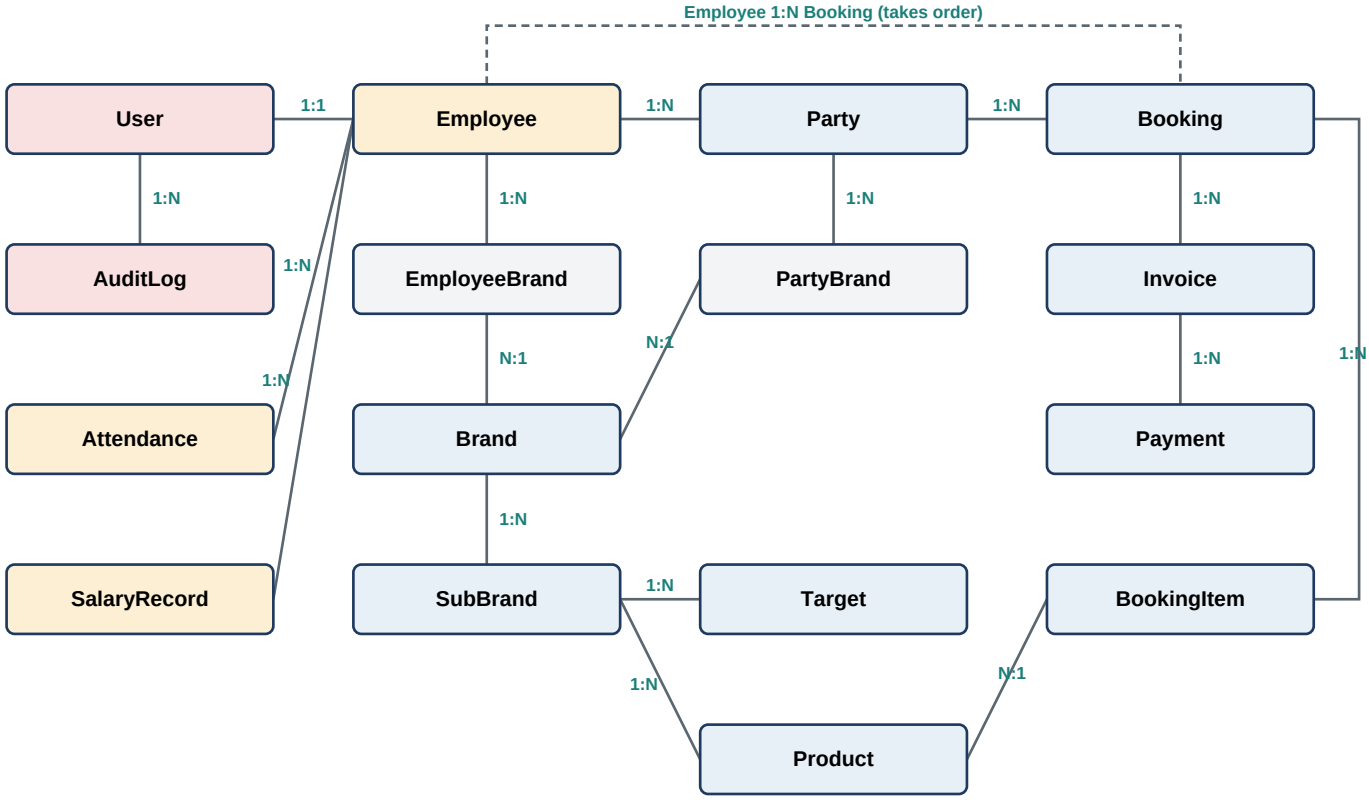

Figure 3 - Entity relationship diagram (main collections)

6.1 Collections and key fields

Collection	Key fields
User	email, passwordHash, role (superAdmin / admin / salesperson), permissions, scope, twoFactorSecret, status, lastLoginAt, failedLogins
Employee	userId, code, name, phone, address, joiningDate, designation, reportingTo, bankDetails (encrypted), documents[], status
Brand	name, code, gstRate, defaultDiscount, status
SubBrand	brandId, name, gender (men/women/kids), status
Product	subBrandId, styleCode, name, season, sizes[], colors[], mrp, dealerPrice, sku/barcode, stock[] (per warehouse/size/color)
EmployeeBrand	employeeId, brandId, subBrandId (optional), role (manager / seller), fromDate, toDate
Party	name, code, city, region, gstNumber, contacts[], creditLimit, paymentTermsDays, outstanding, assignedEmployeeId, status
PartyBrand	partyId, brandId, priceList, discountPct, status
Target	employeeId, brandId, subBrandId, periodType, periodStart, periodEnd, targetValue, targetQty, achievedValue (cached), status
Booking	bookingNo, partyId, employeeId, date, deliveryDate, status, totalAmount, discount, approvalStatus, notes
BookingItem	bookingId, productId, brandId, subBrandId, size, color, qty, rate, discount, amount
Invoice	invoiceNo, bookingId, partyId, date, items[], gstBreakup, total, status, ewayBill
Payment	partyId, invoiceId (optional), amount, mode, referenceNo, date, receivedBy
Attendance	employeeId, date, checkIn, checkOut, status, locationProof

Collection	Key fields
SalaryRecord	employeeId, month, basic, allowances, deductions, advances, incentive, net, status (draft / approved / paid), slipUrl
AuditLog	userId, action, module, recordId, before, after, ip, userAgent, timestamp (append-only)

6.2 Relationship summary

Relationship	Type	Meaning
User - Employee	1:1	Every salesperson has a login; Admins may or may not be employees
Employee - Brand (via EmployeeBrand)	N:M	An employee can manage several brands; a brand has several employees
Brand - SubBrand - Product	1:N:N	Brand contains sub-brands (men's, women's); sub-brands contain products
Party - Brand (via PartyBrand)	N:M	Which party sells which brand, with its own price list and discount
Employee - Party	1:N	Each party is handled by one assigned employee
Employee / Brand / SubBrand - Target	1:N	Target is set per employee and brand (or sub-brand) and period
Party - Booking - BookingItem	1:N:N	A booking belongs to one party and one employee; items hold size, color, qty
Booking - Invoice - Payment	1:N:N	Invoices are raised from bookings; payments reduce the party's outstanding
Employee - Attendance / SalaryRecord	1:N	Monthly records per employee
User - AuditLog	1:N	Every important action is recorded against the acting user

Data rules: money is stored as integer paise; every document has createdAt, updatedAt, createdBy and updatedBy; records are soft-deleted (isDeleted flag) so history and reports stay intact; indexes on partyId, employeeId, brandId, date and status keep reports fast.

7. Security design

An ERP holds salaries, bank details and business financials, so security is a core feature, not an add-on.

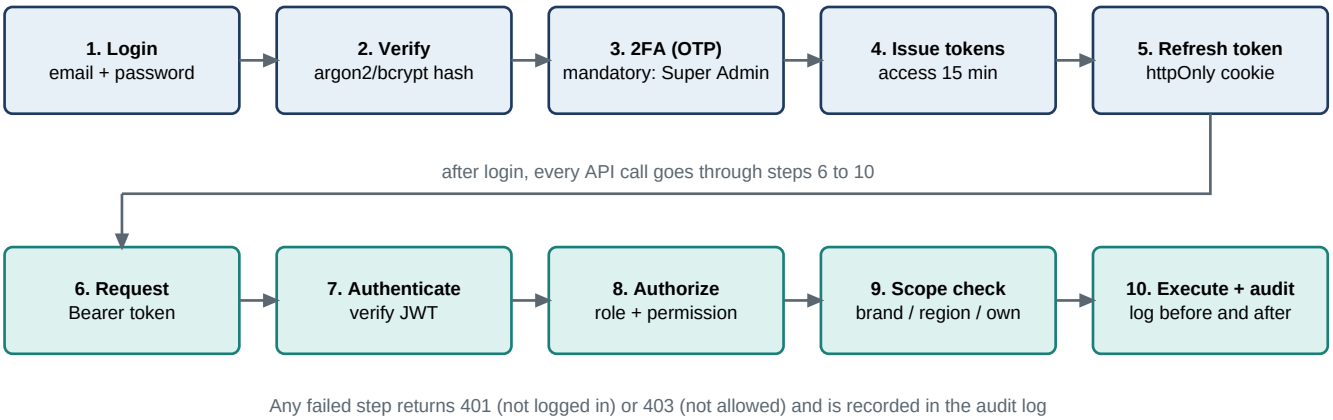

Figure 4 - Authentication and authorization flow

Area	Controls
Authentication	Passwords hashed with argon2 or bcrypt; minimum length and breach-list check; account lockout after repeated failures; mandatory TOTP 2FA for Super Admins and recommended for Admins; short-lived access token (15 min) with rotating refresh token in an httpOnly, secure, SameSite cookie.
Authorization	Role-based access plus per-module permissions plus data scope (brand / region / own). Checked on the server for every request; hiding a button in React is never treated as security.
Super Admin protection	Fixed count of 3; 2-of-3 approval for critical actions; optional IP allowlist; email alert on every Super Admin login from a new device.
Input safety	Schema validation (Zod/Joi) on every endpoint; express-mongo-sanitize against NoSQL injection; output encoding and a strict Content-Security-Policy against XSS.
Transport and headers	HTTPS only (TLS 1.2+), HSTS, helmet headers, CORS allowlist of your own domains, CSRF token where cookies are used.
Abuse prevention	Rate limiting per IP and per user (Redis), stricter limits on login and OTP endpoints, request size limits.
Data protection	Encryption at rest (Atlas), field-level encryption for bank and ID details, signed short-lived URLs for files, secrets in environment variables or a secrets manager, never in Git.
File uploads	Type and size whitelist, antivirus scan if possible, random file names, stored outside the web root.
Audit and monitoring	Append-only AuditLog (who, what, before, after, when, IP); alerts for unusual activity such as bulk export or many failed logins; error tracking with Sentry.
Database access	Least-privilege DB users, network allowlist, no public database port, separate staging and production databases.
Backups and recovery	Daily automated backups with point-in-time recovery, copies stored in another region, restore tested every quarter.
Dependencies and code	npm audit and Dependabot, lockfiles, code review, CI checks for secrets and vulnerabilities.
Privacy and compliance	Collect only needed personal data; role-limited access to salary and ID data; employee data falls under India's data protection law (DPDP Act 2023), so confirm your obligations with a legal or compliance advisor.

8. API design and project structure

REST API, JSON, versioned under /api/v1. Every list endpoint supports pagination, sorting and filters. Every endpoint is protected by authenticate, authorize(module, action) and scope checks except login.

Area	Example endpoints	Who
Auth	POST /auth/login, /auth/refresh, /auth/logout, /auth/2fa/verify	All
Admins	GET/POST /admins, PATCH /admins/:id/permissions	Super Admin
Employees	GET/POST /employees, GET/PATCH /employees/:id, POST /employees/:id/brands	Super Admin, Admin; own via /me
Salary	POST /salary/run, GET /salary/:employeeId, POST /salary/:id/approve	Super Admin, HR Admin
Brands	GET/POST /brands, /brands/:id/sub-brands, /sub-brands/:id/products	Super Admin, Admin; read for sales
Parties	GET/POST /parties, PATCH /parties/:id, POST /parties/:id/brands	Super Admin, Admin; read for sales
Bookings	POST /bookings, PATCH /bookings/:id/status, POST /bookings/:id/approve	All (scoped)
Targets	POST /targets, GET /targets/achievement?employeeId=&brandId;=.=	Super Admin, Admin; read own
Accounts	POST /invoices, POST /payments, GET /parties/:id/ledger, GET /outstanding	Super Admin, Accounts Admin
Reports	GET /reports/brand-sales, /reports/party-sales, /reports/employee-performance, /reports/export	Scoped by role
Audit	GET /audit-logs	Super Admin only

8.1 Suggested folder structure

```
erp/
  server/
    src/
      config/          env, db, redis, logger
      middleware/      auth, rbac, scope, validate, rateLimit, audit, error
      modules/
        auth/  users/  employees/  salary/  attendance/
        brands/  products/  parties/  bookings/  targets/
        invoices/  payments/  reports/  approvals/  notifications/
          (each: model, controller, service, routes, schema)
      jobs/            queues and workers (pdf, reminders, reports)
      utils/           money, dates, pagination
      app.ts  server.ts
    tests/
  client/
    src/
      app/             router, providers, layout
      features/        employees, brands, parties, bookings, targets, reports, admin
      components/      table, form, charts, guards (RequireRole, RequirePermission)
      lib/             api client, auth store, hooks
  docker-compose.yml   .github/workflows/ci.yml   README.md
```

9. Key business rules

Rule	Definition
Target achievement	Achieved value = sum of confirmed booking (or invoiced) amount for the employee, brand/sub-brand and period, net of approved returns. Achievement % = achieved / target x 100. Choose booking-based or invoice-based when you onboard each company and keep it consistent.
Incentive	Slab table per employee grade, for example below 80% = none, 80-100% = x%, above 100% = y%. Slabs are configurable and changes are audited.
Credit control	A booking cannot be confirmed if party outstanding + this booking exceeds the credit limit, unless an authorized approver overrides it (logged).
Discount control	Discount above the brand/party default needs approval; the approver's limit is set per Admin.
Stock	Booking reserves stock by size and color; invoice deducts it; cancellation or return releases or adds it back.
Salary	Monthly run = attendance days x structure + incentive - advances - deductions. Status draft > approved by Super Admin > paid; paid records are locked.
Locking	Closed months and paid salaries cannot be edited, only corrected through an adjustment entry.

10. Build roadmap, testing and deployment

10.1 Phased roadmap

Time estimates assume one full-stack developer using AI coding help and are rough; adjust after the first phase.

Phase	Scope	Rough time
0. Setup	Repo, CI, environments, base project, design system, seed 3 Super Admins	1 week
1. Core	Auth, roles and permissions, audit log; employees; brands and sub-brands; parties and party-brand mapping	3-4 weeks
2. Sales engine	Products and stock, bookings, targets and achievement, brand/party/employee reports, dashboards	3-4 weeks
3. Money	Invoices, payments, outstanding and ageing, salary and attendance, approvals, Tally export	4 weeks
4. Field and alerts	Salesperson PWA, WhatsApp/email notifications, backups hardening, import tool	2-3 weeks
5. Pilot	Run with one real distributor, fix issues, collect feedback before adding extras	2-4 weeks

Show a working version to a real distributor after Phase 2. Their feedback should decide what is built in Phase 3 and beyond.

10.2 Testing strategy

- Unit tests for money, target achievement, incentive and salary calculations (most important).
- API tests for every permission rule: each role must be tested for allowed and denied access.
- End-to-end tests for the main flows: login, create booking, approval, invoice, payment, target report.
- Security tests: injection, broken access control (one admin reading another admin's scope), rate limit and token expiry.
- User acceptance testing with real staff on real (copied) data before go-live.

10.3 Deployment

- Environments: development, staging, production, each with its own database and secrets.
- Containerized with Docker; reverse proxy with HTTPS; CI/CD runs tests then deploys from the main branch.
- Hosting options: a VPS or cloud VM (cheapest, more manual work) or a managed container platform (easier scaling).
- Monitoring: uptime checks, error tracking, log retention, database and disk alerts.

11. Risks and glossary

Risk	Mitigation
Scope creep (ERPs grow endlessly)	Fixed phases; every new request is ranked against the pilot feedback
Wrong calculations (targets, salary, tax)	Heavy unit tests; sample data checked by an accountant; locked months
Data leak or misuse by insiders	Scoped permissions, audit logs, 2FA, alerts on exports
Competing established products (Tally, Busy, Zoho, ERPNext)	Differentiate through apparel-specific features: size-color stock, brand-wise targets, party-brand mapping, salesperson app
Adoption by staff	Simple screens, training, Hindi labels if needed, import of existing Excel data

Glossary

Term	Meaning
ERP	Enterprise Resource Planning: one connected system for a company's operations
RBAC	Role-Based Access Control: permissions granted by role
JWT	JSON Web Token: signed token proving a user is logged in
2FA / TOTP	Second login step using a one-time code from an authenticator app
Party	A dealer or retailer who buys from the distributor
Booking	An order taken from a party before dispatch and invoicing
PWA	Progressive Web App: a website that installs and behaves like a mobile app
Audit log	A permanent record of who did what and when
Soft delete	Marking a record as deleted without erasing it